COLLÈGE LA CITÉ · SCIENCE DES DONNÉES APPLIQUÉES

Bases de données massives avancées (IFM30522)

Projet d'équipe (UA2) · Rapport d'évaluation

Tableau de bord des ventes multi-magasins

24 000 ventes de 2025 déposées dans HDFS, agrégées par MapReduce Python (Hadoop Streaming) sur un cluster Hadoop de 3 nœuds, restituées dans une application Streamlit filtrable et distribuées sous la forme d'une image Docker publiée.

Remis par

Yacine Lezoul · 2748526

Adel Benhamma · 2741167

Barnabe Landry Evouna · 2751210

Mouncif Bouchachi · 2743259

Rimy Meziani · 2741840

Douyeri Lamoussa Sanogo · 2731042

Professeur	Oussama Naciri
Cours	Bases de données massives avancées (IFM30522-0030-A2026)
Date de remise	9 novembre 2026
Dépôt GitHub	https://github.com/lezoul/hadoop-mapreduce-ventes-multimagasins
Image Docker	yacinelezoul/dashboard-ventes:1.0 (Docker Hub)
Documents	Ce rapport suit la grille d'évaluation. Le **rapport détaillé** (`rapport_detaille.pdf`) contient le guide de reproduction commande par commande, le code commenté et toutes les preuves ; toutes les commandes, commentées et dans l'ordre, sont regroupées dans `scripts_execution/commandes_execution.md` .

SECTION 01

Contexte, architecture et grille d'évaluation

Une enseigne fictive de **8 magasins dans 6 villes** veut comparer ses magasins, suivre son chiffre d'affaires (CA) et repérer ses meilleurs produits. Nous avons construit la chaîne complète : contrôle du CSV, dépôt dans HDFS, calcul par MapReduce Python, tableau de bord qui lit uniquement les sorties MapReduce, puis image Docker publiée.

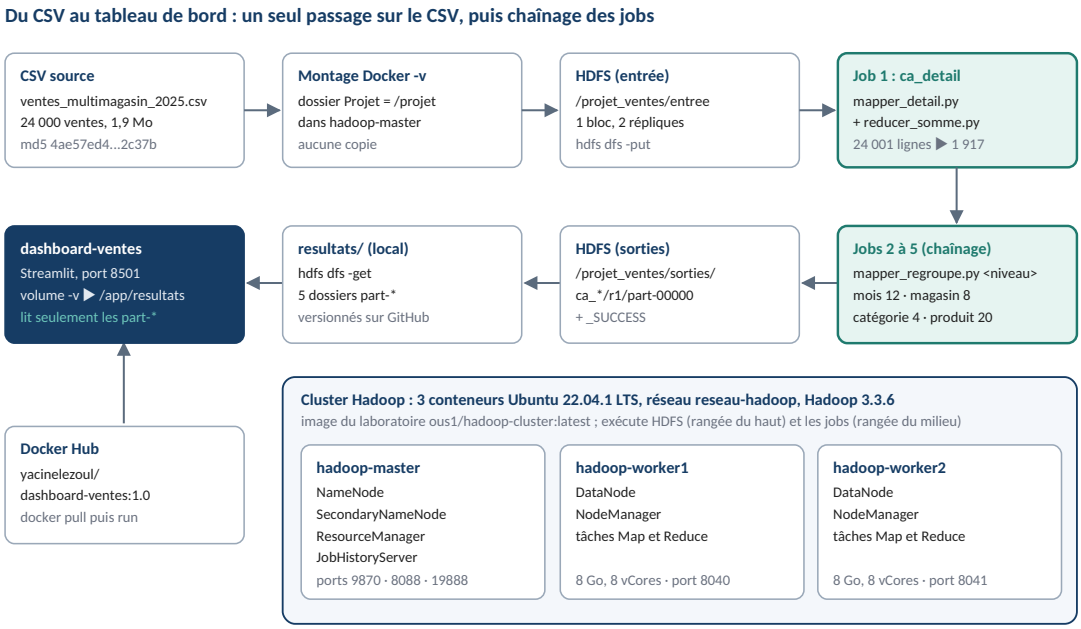


Figure 1. *Architecture : stockage HDFS, job détaillé, jobs chaînés, récupération des sorties et application conteneurisée*

Choix d'architecture. Un seul job lit le CSV et calcule le CA au grain le plus fin (mois, magasin, ville, catégorie, produit) : `ca_detail`, 1 917 lignes. Les 4 analyses demandées sont produites par 4 jobs chaînés sur ce résultat, au lieu de relire 4 fois les 24 000 ventes. Le même `ca_detail` alimente les filtres du tableau de bord.

Environnement. Image du laboratoire `ous1/hadoop-cluster:latest` : Ubuntu 22.04.1 LTS, Hadoop 3.3.6, Java 1.8, Python 3.10.12 ; 1 maître (`hadoop-master` : NameNode, ResourceManager, JobHistoryServer) et 2 workers (`hadoop-worker1`, `hadoop-worker2` : DataNode, NodeManager) sur le réseau Docker `reseau-hadoop`. Tableau de bord : Streamlit 1.66.1, Plotly 7.1.0, pandas 3.1.0.

Critère de la grille	Pts	Où nous y répondons
Compréhension des données, contrôles et dépôt HDFS	4	Section 02
Hadoop Streaming / MapReduce Python et exactitude	9	Section 03
Mappers, Reducers, partitions, YARN et tests	4	Section 04
Tableau de bord Python	5	Section 05
Image Docker construite, publiée, exécutable, documentée	4	Section 06, `preuve_docker.txt`
Rapport : clarté, architecture, preuves, traçabilité	3	Sections 01 et 07, rapport détaillé
PowerPoint et démonstration	3	Section 08, `presentation.pptx` (12 diapositives)
Participation et réponses aux questions	3	Section 08 (répartition, 12 questions)

SECTION 02 · CRITÈRE 4 POINTS

Données, contrôles de qualité et dépôt HDFS

2.1 Contrôles de qualité (`sources/controle_qualite.py`)

Contrôle	Résultat
Structure	24 000 lignes hors en-tête, 12 colonnes partout, en-tête conforme au dictionnaire
Valeurs manquantes, types	0 valeur manquante ; dates de 2025 au format AAAA-MM-JJ ; quantités entières positives ; prix de 6,50 à 219,00 $
Valeurs catégorielles	8 magasins, 6 villes, 20 produits, 4 catégories ; VALIDEE 22 576, ANNULEE 1 424 ; CARTE 17 246, ESPECES 5 304, VIREMENT 1 450
Doublons	0 doublon d'identifiant ; 334 lignes identiques hors identifiant, **conservées** : deux clients peuvent acheter le même produit le même jour, et chaque ligne a un identifiant unique
Point de vigilance	1 749 ventes ont un montant net qui tombe sur un demi-cent : la règle d'arrondi change le total

2.2 Règles de calcul

Pour chaque vente **VALIDEE** : montant net = quantité × prix unitaire × (1 − remise / 100), **arrondi au cent avant l'agrégation** (demi-cent arrondi vers le haut). Les ventes ANNULEE sont exclues. Le calcul est fait avec `Decimal` et `ROUND_HALF_UP`, et les montants circulent en **cents entiers** : en flottant, `round(6.50 × 0.95, 2)` donne 6,17 au lieu de 6,18, et le total de l'année serait faux.

2.3 Cluster vérifié et dépôt dans HDFS

Vérification	Commande et résultat
Services	`jps` : NameNode, SecondaryNameNode, ResourceManager, JobHistoryServer sur le maître ; DataNode et NodeManager sur chaque worker
HDFS et YARN	`hdfs dfsadmin -report` : Live datanodes (2) ; `yarn node -list` : 2 nœuds RUNNING
Organisation HDFS	`/projet_ventes/entree` (CSV), `entree_blocs_1m` (copie à blocs de 1 Mo pour l'expérience Mappers), `tests`, `sorties/<job>/r<nombre de Reducers>`
Fichier cohérent	md5 local = md5 HDFS = `4ae57ed472eb5c10dd07814deba2c37b` ; 24 001 lignes des deux côtés ; 1 985 795 octets
Blocs et répliques	`hdfs fsck -files -blocks -locations` : 1 bloc (2 blocs de 1 048 576 et 937 219 octets pour la copie à 1 Mo) ; 2 répliques sur les 2 workers ; Status HEALTHY

SECTION 03 · CRITÈRE 9 POINTS

Traitements MapReduce Python et exactitude

3.1 Paires clé-valeur et justification

Job	Mapper	Clé émise	Sortie
1 · `ca_detail`	`mapper_detail.py` sur le CSV	`mois|magasin|ville|categorie|id_produit|nom_produit`	1 917 lignes
2 · `ca_mois`	`mapper_regroupe.py <niveau>` sur `ca_detail`	`mois`	12 lignes
3 · `ca_magasin`		`magasin|ville`	8 lignes
4 · `ca_categorie`		`categorie`	4 lignes
5 · `ca_produit`		`id_produit|nom_produit|categorie`	20 lignes
Contrôle	`mapper_mois.py` sur le CSV	`mois`	12 lignes

La **valeur** est toujours le montant net en cents. La clé est le critère de regroupement : toutes les valeurs d'une même clé sont additionnées. Le **Reducer est le même pour les 6 jobs** (`reducer_somme.py`) : il reçoit les lignes triées par clé et écrit une somme à chaque changement de clé. Exemple : la vente V000001 donne `2025-08|M05|Toronto|Bureau|P007|Lot de stylos → 801`. Le Mapper ignore l'en-tête, les lignes mal formées et les ventes ANNULEE.

3.2 Exécution réelle par Hadoop Streaming

hadoop-master : job 1 (les jobs 2 à 5 changent -files, -mapper, -input et -output)

```
mapred streaming -D mapreduce.job.name="ca_detail_r1" -D mapreduce.job.reduces=1 \
  -files /projet/sources/mapper_detail.py,/projet/sources/reducer_somme.py \
  -mapper "python3 mapper_detail.py" -reducer "python3 reducer_somme.py" \
  -input  /projet_ventes/entree/ventes_multimagasin_2025.csv \
  -output /projet_ventes/sorties/ca_detail/r1
```

Compteurs du job 1 : Map input records 24 001, Map output records 22 576 (les 1 424 annulées n'ont rien émis), Reduce output records 1 917. Les 13 jobs du projet sont à l'état SUCCEEDED dans YARN (`application_1793588531268_0001` à `_0013`). Chaque sortie est dans son propre dossier HDFS, puis copiée par `hdfs dfs -get` dans `resultats/` pour le tableau de bord.

3.3 Exactitude

La somme de **chaque** sortie vaut **194 661 314 cents, soit 1 946 613,14 $**, quel que soit le job ou le nombre de Reducers. Le CA mensuel obtenu par chaînage et celui calculé directement depuis le CSV par un job indépendant ont le **même md5** (`db30e50fbe9042172383b551c5a4a634`).

CA mensuel	$	CA par magasin	$	Catégorie / Top 5 produits	$
janvier	123 316,69	M05 Toronto	377 268,41	Electronique (40,9 %)	796 377,63
mars	143 888,51	M01 Ottawa	276 695,18	Maison (32,3 %)	627 786,89
juin	140 922,88	M03 Montréal	244 011,03	Sport (16,4 %)	320 095,04
septembre	159 640,31	M06 Québec	222 651,74	Bureau (10,4 %)	202 353,58
novembre	211 219,00	M04 Montréal	217 282,75	1. Ecran 24 pouces	266 676,30
décembre	228 992,72	M08 Kingston	214 109,03	2. Aspirateur compact	225 395,30
Croissance de +86 % entre janvier et décembre		M07 Gatineau	206 204,68	3. Casque audio	193 918,15
		M02 Ottawa	188 390,32	4. Cafetiere / 5. Webcam HD	151 896,30 / 142 774,10

Les 12 mois, les 8 magasins, les 2 villes à deux magasins (Ottawa 465 085,50 $, Montréal 461 293,78 $) et les 20 produits sont dans le tableau de bord et dans `resultats/`.

SECTION 04 · CRITÈRE 4 POINTS

Mappers, Reducers, partitions, YARN et tests

4.1 Nombre de Reducers et partitions

Reducers (job `ca_mois`)	Durée	Shuffled Maps	Mémoire totale	Fichiers de sortie
1	23 s	2	0,95 Go	`part-00000` : 12 mois
2	27 s	4	1,23 Go	2 fichiers : 7 et 5 mois
4	34 s	8	1,73 Go	00000 : 01, 05, 09, 12 · 00001 : 02, 06 · 00002 : 03, 07, 10 · 00003 : 04, 08, 11

Shuffle et partitions. Chaque paire reçoit la partition `hash(clé) mod nombre de Reducers` : une même clé va toujours au même Reducer, et comme il y a moins de Reducers que de clés, plusieurs mois partagent un fichier. Chaque Reducer copie les sorties de chaque Mapper (Shuffled Maps = Mappers × Reducers), puis les trie par clé. Le total reste 194 661 314 dans les 3 cas. Sur 12 clés, ajouter des Reducers ajoute des conteneurs à démarrer et des copies sans travail utile : la durée augmente. La ligne `Killed reduce tasks=1` du job à 4 Reducers vient de l'exécution spéculative (copie de secours d'une tâche lente), pas d'une erreur.

4.2 Nombre de Mappers et InputSplits

Configuration (job 1 sur le CSV de 1,99 Mo)	Mappers	Durée
`mapreduce.job.maps=1` , fichier à 1 bloc	1	14 s
Par défaut (Streaming demande 2), fichier à 1 bloc	2	39 s
`mapreduce.job.maps=4` , fichier à 1 bloc	4	21 s
`mapreduce.job.maps=1` , fichier à 2 blocs de 1 Mo	2	17 s

Un Mapper traite un InputSplit. `mapreduce.job.maps` n'est qu'une indication : on peut couper un bloc en plusieurs splits, mais pas descendre sous le nombre de blocs. Le nombre de DataNodes ou de répliques n'entre pas en compte. Un split est ajusté aux fins de ligne : la vente V012677, coupée à l'octet 1 048 576, est lue entière par le premier Mapper.

4.3 Ressources YARN

Chaque NodeManager déclare **8 192 Mo et 8 vCores** ; un conteneur reçoit entre 1 024 Mo / 1 vCore (minimum) et 8 192 Mo / 4 vCores (maximum). Chaque tâche a reçu **1 024 Mo et 1 vCore**, pour un pic réel d'environ 357 Mo par Map. Le job à 4 Mappers a consommé 130 124 Mo·s et 90 vCore·s ; l'ApplicationMaster tournait sur hadoop-worker2. **Limite** : YARN déclare 16 Go au total alors que Docker dispose de 7,47 Go, une surréservation sans effet sur nos petits jobs.

4.4 Cas de test vérifié à la main

Fichier `tests/echantillon_test.csv` de 7 ventes, dont 2 de même clé (T01, T02), un demi-cent (T03 : 6,175 $ arrondi à 6,18 $), une vente ANNULEE (T04) et des mois, magasins et catégories différents. Résultat attendu calculé à la main : 4 clés, **77 348 cents**. Compteurs Hadoop : Map input 8, Map output 5, Reduce output 4. La commande `diff` entre la sortie et le résultat attendu ne renvoie rien en local, sur Hadoop et après chaînage : **3 tests réussis**.

SECTION 05 · CRITÈRE 5 POINTS

Tableau de bord Python



Figure 2. *Filtre Ottawa + Gatineau : indicateurs recalculés (671 290 $, 34,5 % du CA annuel, +88 %) et courbe mensuelle de la sélection*

Exigence	Réalisation (`application_python/app.py`)
4 analyses obligatoires	Onglet Analyses : CA mensuel (courbe), CA par magasin et par ville (barres), CA par catégorie (anneau), Top 5 produits (barres)
Au moins 2 filtres	**4 filtres** : ville, magasin (limité aux villes choisies), catégorie, période ; tous les graphiques et indicateurs se recalculent
Indicateurs	CA de la sélection et part du total, évolution janvier à décembre, meilleur mois, meilleur magasin
Tableau	Tableau croisé : villes, magasins et catégories en accordéons, mois en colonnes, totaux
Unités, période, provenance	Montants en $ CAD hors taxes ; période 2025 rappelée ; onglet Provenance : dossier lu, fichiers `part-*`, nombre de lignes et total de contrôle de chaque job
Pas de lecture du CSV	Le seul module de lecture, `lecture_resultats.py`, ne lit que des fichiers `part-*` ; l'image Docker ne contient aucune donnée

Pourquoi lire `ca_detail`. Une première version ne lisait que les 4 sorties agrégées : impossible de filtrer le CA mensuel par ville, puisque `ca_mois` ne contient plus la ville. En lisant `ca_detail`, déjà agrégé par MapReduce (1 917 lignes au lieu de 24 000 ventes), le tableau de bord ne fait qu'additionner des lignes filtrées. Les 4 sorties agrégées servent de contrôle dans l'onglet Provenance : leurs totaux sont identiques à ceux de `ca_detail`.

SECTION 06 · CRITÈRE 4 POINTS

Image Docker construite, publiée et exécutable

Élément	Valeur
Image et tag	`yacinelezoul/dashboard-ventes:1.0` sur Docker Hub (public) : hub.docker.com/r/yacinelezoul/dashboard-ventes
Dockerfile	`python:3.14-slim` ; `requirements.txt` installé avant le code (cache : build de 172 s, puis 1,4 s) ; port 8501 ; résultats lus dans `/app/resultats`
Publication	`docker push` : digest `sha256:412f047f…71caa7` ; 190 Mo compressée sur Docker Hub
Preuve	Image locale supprimée (`docker rmi`), puis `docker pull` (même digest) et `docker run` ; l'onglet Provenance affiche `/app/resultats` ; détail dans `preuve_docker.txt`
Sécurité	Connexion à Docker Hub par le navigateur ; aucun mot de passe ni jeton dans l'image, le code ou le dépôt

Hôte : récupérer et lancer l'image (depuis la racine du dépôt cloné)

```
docker pull yacinelezoul/dashboard-ventes:1.0
docker run -d --name dashboard-ventes -p 8501:8501 \
  -v "$(pwd)/resultats:/app/resultats:ro" yacinelezoul/dashboard-ventes:1.0
# puis ouvrir http://localhost:8501
```

Le volume `-v` donne au conteneur les sorties MapReduce en lecture seule ; sous PowerShell, remplacer `\` par `` ` `` et `$(pwd)` par `${PWD}`.

SECTION 07 · CRITÈRE 3 POINTS

Preuves, traçabilité et difficultés

7.1 Où trouver chaque preuve

Étape	Journal (`scripts_execution/journaux/`)	Capture (`captures/`)
Cluster et services	01, 02	01 Docker, 02 NameNode, 03a et 03b DataNodes, 05 YARN
Dépôt HDFS et fsck	03	04 dossiers de sortie HDFS
Jobs MapReduce	04, 08	06 les 13 jobs SUCCEEDED
Reducers, Mappers, YARN	05, 06, 07	05 ressources des nœuds
Tableau de bord	09	07a, 07b, 08, 09, 09b, 10
Cas de test	10	
Image Docker	11 et `preuve_docker.txt`	11 application lancée depuis l'image, 12 page Docker Hub

Toutes les commandes sont dans `scripts_execution/commandes_execution.md`, dans l'ordre d'exécution, chacune expliquée, avec l'endroit où la taper et le résultat attendu. Le rapport détaillé les explique une par une (annexe A), donne le code commenté (annexe B) et les captures légendées (annexe C).

7.2 Difficultés rencontrées et résolution

Difficulté	Symptôme	Résolution
Arrondi au demi-cent	Premier total calculé en flottant : 194 660 289 cents, soit 10,25 $ d'écart	`Decimal` + `ROUND_HALF_UP`, montants en cents entiers ; vérifié par le cas de test (T03)
Nombre de Mappers	2 Mappers pour un fichier d'un seul bloc, alors que nous en attendions 1	Lecture des compteurs : Streaming demande 2 Mappers par défaut ; expérience de la section 4.2
Filtres du tableau de bord	Les sorties agrégées ne permettent pas de croiser ville et mois	Tableau de bord alimenté par `ca_detail`, sorties agrégées utilisées comme contrôle
Fins de ligne Windows	Git aurait converti les fins de ligne et changé le md5 du CSV	`.gitattributes` : scripts en LF, CSV jamais converti
Limites	Volume faible : le démarrage des conteneurs domine les durées, une seule mesure par configuration ; surréservation de la mémoire YARN (section 4.3)	

SECTION 08 · CRITÈRES 3 + 3 POINTS

Répartition des tâches et soutenance

Membre	Contribution	Partie présentée
Yacine Lezoul	Coordination ; cluster, HDFS et YARN ; contrôle qualité ; code MapReduce et chaînage ; expériences ; tests ; tableau de bord ; image Docker ; dépôt GitHub	Diapositives 1 et 2 : contexte, architecture
Mouncif Bouchachi	Reproduction de la chaîne à partir du README, avec captures	3 et 4 : environnement, données, HDFS
Adel Benhamma	Relecture du rapport ; vérification des réponses aux 12 questions	5 et 6 : MapReduce, chaînage, résultats
Douyeri Lamoussa Sanogo	Mise en forme et répétition du PowerPoint ; chronométrage de la démonstration	7 et 8 : Reducers, Mappers, YARN
Rimy Meziani	Réexécution du cas de test sur sa machine	9 et 10 : tests, tableau de bord
Barnabe Landry Evouna	Lancement de l'image depuis Docker Hub sur sa machine	11 et 12 : Docker, démonstration, conclusion

Démonstration (de HDFS au tableau de bord) : `docker ps` et `jps` ; `hdfs dfs -ls -R /projet_ventes` et `hdfs fsck` ; lancement en direct du job `ca_categorie` suivi sur http://localhost:8088 ; contenu des 4 partitions de `ca_mois/r4` ; `docker pull` puis `docker run` de l'image publiée, 2 filtres et onglet Provenance. Plan de secours : captures et journaux.

8.1 Réponses aux 12 questions de compréhension

#	Réponse courte
1	HDFS découpe le fichier en blocs répliqués : il survit à la perte d'un nœud et chaque bloc est traité là où il est stocké, en parallèle. Un CSV local dépend d'un seul disque.
2	Bloc : découpage physique du stockage (128 Mo par défaut). InputSplit : découpage logique de la lecture, ajusté aux fins de ligne. Tâche Mapper : programme qui traite un InputSplit.
3	Non : il dépend du nombre d'InputSplits (taille du fichier, des blocs, paramètres). Avec 2 DataNodes et 2 répliques, nous avons eu 1, 2 ou 4 Mappers.
4	Streaming distribue les scripts (`-files`), envoie les lignes sur l'entrée standard du Mapper, coupe clé et valeur sur la tabulation, fait le Shuffle & Sort et envoie au Reducer les lignes triées.
5	La clé est le critère de regroupement : `2025-03` donne le CA de mars, `M05|Toronto` celui d'un magasin. Une clé fine (`ca_detail`) permet de reconstituer tous les regroupements plus larges.
6	Chaque paire reçoit la partition `hash(clé) mod R` ; le Shuffle copie chaque partition vers son Reducer depuis tous les Mappers, le Sort trie par clé : les valeurs d'une clé arrivent à la suite.
7	Le hash est déterministe, donc une clé va toujours dans la même partition ; il y a moins de Reducers que de clés, donc plusieurs clés partagent un Reducer (4 fichiers pour 12 mois).
8	1, 2 puis 4 fichiers `part-*` ; Shuffle de 2, 4 puis 8 copies ; total inchangé. Sur notre petit volume, la durée passe de 23 à 34 s : chaque Reducer est un conteneur à démarrer.
9	`yarn.nodemanager.resource.memory-mb` et `cpu-vcores` (capacité d'un nœud), `yarn.scheduler.minimum/maximum-allocation-mb` et `-vcores` (bornes d'un conteneur), `mapreduce.map/reduce.memory.mb` (demande des tâches).
10	Le Mapper n'émet rien si le statut n'est pas VALIDEE : Map output 22 576 pour 24 000 ventes ; la vente annulée du cas de test est absente du résultat.
11	`lecture_resultats.py` ne lit que des `part-*` ; l'image ne contient aucune donnée ; l'onglet Provenance liste les fichiers lus et leurs totaux, identiques au cent près aux sorties MapReduce.
12	L'image est un modèle figé, identifié par son digest et stocké dans le registre ; le conteneur est une instance en cours d'exécution, avec ses ports et ses volumes. Une image peut lancer plusieurs conteneurs.